AUTONOMOUS EDGE-AI FOR DDOS AND BRUTE-FORCE DETECTION AND AUTOMATED IP MITIGATION IN RESOURCE-CONSTRAINED REMOTE ENTERPRISE NETWORKS

MOHAMMAD SYARAFUDDIN BIN MOHD ZAIDI

A project report submitted in partial fulfilment of the

requirements for the award of the degree of

Bachelor of Information Technology (Hons)

Faculty of Science, Management, and Computing

Universiti Teknologi PETRONAS

July 2026

ABSTRACT

Remote industrial and operational sites, including offshore platforms and isolated agricultural facilities, may depend on satellite or cellular links with high latency, limited bandwidth, or intermittent availability, reducing the responsiveness of cloud dependent detection and mitigation. Transmitting packet, flow, or security event data to remote cloud-based intrusion detection systems IDSs risks increasing the interval between attack detection and mitigation before remote actions are applied. Computationally intensive detection models may be impractical on resource constrained edge devices due to processing, memory, power, and cost constraints, which can spike CPU and memory utilisation during high volume distributed denial of service DDoS attacks. An autonomous edge machine learning gateway is designed for deployment in line at the network demarcation point to monitor network flows and apply local filtering actions. The proposed gateway uses lightweight edge machine learning, specifically a random forest classifier trained on eight selected network flow features from the CIC IDS 2017 dataset targeting benign, SYN flood DDoS, and SSH brute force traffic. A real time flow aggregation component reproduces feature definitions locally to reduce model size, inference latency, and resource utilisation on a Raspberry Pi 4 acting as an in-line gateway. To reduce user space overhead after detection during controlled high-volume simulations and protect downstream devices, an asynchronous process installs temporary filtering rules via the Linux iptables or netfilter framework. When the classifier identifies a suspicious pattern in user space, it automatically installs a temporary source IP filtering rule to filter subsequent matching packets before further user space processing is required. Using a controlled network testbed and authorised simulated attack scenarios, a trade off analysis of performance, latency, and resource usage will compare the reduced feature model against a full feature or simpler baseline configuration via precision, recall, F1 score, false positive rate by class, CPU utilisation, RAM utilisation, throughput, packet loss, inference latency, and time to block. The study examines if the design can deliver local automated detection and temporary IP-based filtering to replace cloud-dependent systems and delayed remote mitigation. The model successfully reduces local overhead and safeguards downstream assets, but upstream saturation persists as a key limitation.

TABLE OF CONTENTS

LIST OF TABLES V

LIST OF FIGURES VI

LIST OF SYMBOLS / ABBREVIATIONS VII

LIST OF APPENDICES VIII

1. INTRODUCTION 1

1.1 BACKGROUND 1

1.2 PROBLEM STATEMENTS 1

1.3 AIMS AND OBJECTIVES 3

1.3.1 Research Questions 4

1.4 SCOPE OF STUDY 4

1.5 SIGNIFICANCE OF THE PROJECT 5

2. LITERATURE REVIEW 7

2.1 THE SHIFT FROM CLOUD TO EDGE COMPUTING IN CYBERSECURITY 7

2.2 MACHINE LEARNING FOR TRAFFIC ANALYSIS AND ANOMALY DETECTION 7

2.2.1 Types of Machine Learning Algorithm for IDS 8

2.2.2 The Random Forest Advantages 8

2.3.1 Definition and Industry Applications of Lightweight Edge-ML 9

2.3.2 Comparative Analysis of Lightweight Edge-ML with other Traditional Edge ML 9

2.4 RESOURCE-CONSTRAINED REMOTE ENTERPRISE NETWORKS 10

2.4.1 Characteristics and Examples 10

2.5 COMPARATIVE SUMMARY OF REVIEWED STUDIES 11

2.6 CRITICAL ANALYSIS AND IDENTIFIED RESEARCH GAPS 12

3. METHODOLOGY 15

3.1 SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) 15

3.1.1 Overview of the Agile Methodology 15

3.1.2 Application of SDLC Phases to the Project 16

3.2 PHASE 1: PLANNING AND SYSTEM ARCHITECTURE 16

3.2.1 Overall System Architecture and Network Demarcation 16

3.2.2 Hardware and Software 17
3.3 PHASE 2: DESIGN AND IMPLEMENTATION 18
3.3.1 Dataset Acquisition and Feature Engineering 18
3.3.2 Model Development and Baseline Benchmarking 19
3.3.3 Edge Integration and Automated Firewall Rules 19
3.4 PHASE 3: TESTING AND EVALUATION PLAN 20
3.4.1 Threat Simulation Scenarios 20
3.4.2 Performance Metrics 20
3.5 PROJECT TIMELINE 21
4. EXPECTED OUTCOMES 22
4.1 WORK COMPLETED IN FYP1 22
4.1.1 Dataset Preparation and Feature Reduction 22
4.1.2 Architecture Design and SDLC Planning 22
5. CONCLUSION AND RECOMMENDATIONS 23
5.1 SUMMARY 23
5.2 CONCLUSION OF FYP1 23
5.3 PROJECT CONTINUATION IN FYP2 23
5.4 RECOMMENDATION FOR FUTURE WORK 24
REFERENCES 25
APPENDICES 27

LIST OF TABLES

Table 1: Comparative Summary of Reviewed Studies…….....11

LIST OF FIGURES

Figure 1: Latency Comparison and Vulnerability Window……2

Figure 2: Conceptual Diagram of the Edge-AI Pre-Filter Shield at the Network Demarcation Point…….6

Figure 3: Random Forest Ensemble Architecture for Network Traffic Classification…9

Figure 4: Traditional Deep Learning Edge Deployments vs. Lightweight Edge-ML...10

Figure 5: Adapted Agile Software Development Life Cycle for Edge-ML Integration……...15

Figure 6: Multi-Zone Network Demarcation and Edge Gateway System Architecture…….17

Figure 7: Real-Time Flow Aggregation and Feature Extraction Pipeline……18

Figure 8: Closed-Loop Linux Kernel Mitigation and Packet Inspection Execution Flow……20

Figure 9: Conceptual Edge-ML Gateway Dashboard Mockup for Real-Time Telemetry and Mitigation…….24

LIST OF SYMBOLS / ABBREVIATIONS

ANN	Artificial Neural Network
DDoS	Distributed Denial of Service
GEO	Geostationary Earth Orbit
IDS	Intrusion Detection System
IP	Internet Protocol
IPv4	Internet Protocol version 4
ISP	Internet Service Provider
ML	Machine Learning
OT	Operational Technology
PLC	Programmable Logic Controller
RAM	Random Access Memory
RF	Random Forest
SDLC	Software Development Life Cycle
SMOTE	Synthetic Minority Over-sampling Technique
SSH	Secure Shell
SVM	Support Vector Machine
SYN	Synchronize (TCP Flag)
TinyML	Tiny Machine Learning
USB	Universal Serial Bus
WAN	Wide Area Network
XAI	Explainable Artificial Intelligence

LIST OF APPENDICES

Appendix A: FYP1 Project Gantt Chart…….....27

Appendix B: FYP2 Project Gantt Chart……...27

Appendix C: Trade-Off Conceptual Model: Edge-AI vs Cloud IDS……28

1. INTRODUCTION

1.1 Background

Remote enterprise environments, such as offshore rigs and isolated agricultural facilities, may depend on satellite or cellular links that can experience high latency, limited bandwidth, or intermittent availability. Modern Intrusion Detection System (IDS) architectures generally fall into three categories: cloud-managed systems that rely on remote servers for heavy analysis, locally deployed systems constrained by edge hardware, and hybrid designs that attempt to balance the two. In cloud-managed and hybrid setups, systems must transmit network data for machine learning processing and signature matching. While some systems transmit raw packets, others are designed to send flow records, extracted features, or security alerts to remote cloud servers (Cherukuri, 2024; Ali et al., 2025). Furthermore, while these remote sites rely on Internet Service Providers (ISPs) for connectivity and broad Wide Area Network (WAN) volumetric traffic scrubbing, ISPs generally have limited visibility into encrypted, local, or organisation-specific behaviour at the customer edge. Consequently, a critical security gap exists at the network demarcation point, requiring an offline defense mechanism capable of local detection, downstream protection, and rapid filtering without continuous cloud connectivity.

1.2 Problem Statements

Current network security architectures face three critical limitations when applied to remote enterprise environments.

1. **Cloud-Dependent Latency and the Vulnerability Window**
    - **Affected Environment & Supporting Evidence:** Remote sites utilizing satellite and cellular links inherently experience high transmission latency, with typical round-trip propagation delays ranging from 200 ms to nearly 500 milliseconds for Geostationary Earth Orbit (GEO) connections (Zhu & Jiang, 2022).
    - **Practical Consequence:** This physical transmission latency severely reduces the responsiveness of cloud-managed detection systems

(Cherukuri, 2024). During this delay, a successful credential compromise (e.g., via a targeted brute-force attack) may create an opportunity for further intrusion, subject to internal network controls, long before remotely generated mitigation actions can be applied, as illustrated in Figure 1.

- **Specific Unresolved Gap:** There is an absence of edge-native security frameworks capable of eliminating this communication delay through immediate, offline automated mitigation at the local level.

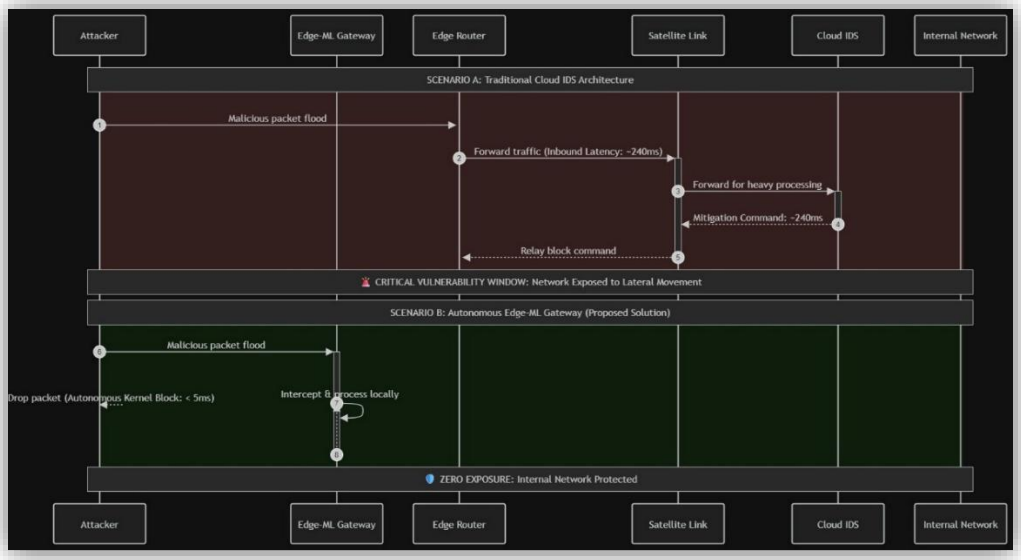


Figure 1: Latency Comparison and Vulnerability Window

2. Limited ISP Visibility and Upstream Saturation at the Customer Edge

- **Affected Environment & Supporting Evidence:** ISPs provide volumetric traffic scrubbing on the wider WAN, but they inherently have limited visibility into encrypted, local, or organisation-specific behaviour directly at the customer edge router (Aryee, 2026).
- **Practical Consequence:** Targeted attacks, such as localized DDoS vectors or SSH brute-force attempts, can bypass broad WAN filters and strike the local network. While edge-based mitigation cannot prevent external volumetric attack traffic from consuming upstream satellite or cellular bandwidth, a failure to filter these malicious packets at the demarcation point leaves downstream internal corporate firewalls and endpoints highly vulnerable to rapid CPU exhaustion.

- **Specific Unresolved Gap:** Current architectures lack a localized pre-filter shield explicitly engineered for local detection, downstream protection, and the rapid filtering of malicious traffic before it impacts the internal corporate network.

3. **Heavy Computational Demands on Resource-Constrained Hardware**
    - **Affected Environment & Supporting Evidence:** Modern, computationally intensive intrusion-detection models require substantial memory, processing power, and often dedicated Graphics Processing Units (GPUs) to evaluate complex traffic patterns (Ibrahim et al., 2026; Mohale & Obagbuwa, 2025).
    - **Practical Consequence:** Deploying these heavy systems is physically and financially impractical on resource-constrained edge devices commonly used in remote industrial deployments (e.g., a standalone Raspberry Pi). Unoptimized models may severely increase CPU and memory utilisation, leading to self-inflicted resource exhaustion during high-volume DDoS floods (Hafid et al., 2024).
    - **Specific Unresolved Gap:** There remains a critical lack of frameworks that seamlessly integrate lightweight detection with instantaneous, offline mitigation specifically tailored to operate within the strict processing, memory, and power constraints of edge hardware (Amuthadevi et al., 2025).

1.3 Aims and Objectives

The primary aim of this project is to propose and evaluate an autonomous edge-machine-learning gateway designed to act as an in-line security sentinel for resource-constrained environments. The specific objectives are:

1. To develop a lightweight edge-ML model capable of classifying SYN-flood DDoS and SSH brute-force traffic with high precision, recall, and F1-score on resource-constrained hardware.
2. To design and deploy an autonomous in-line edge gateway on a Raspberry Pi that integrates real-time flow aggregation with the local Linux iptables/netfilter framework to instantly isolate detected threats.

3. To evaluate the gateway's performance by measuring detection accuracy, mitigation latency (time-to-block), and hardware resource consumption (CPU and RAM utilisation) under simulated attack conditions.

1.3.1 Research Questions

To achieve the defined objectives and ensure a one-to-one alignment with the evaluation measures, this study addresses the following research questions:

- **RQ1:** What is the detection performance (precision, recall, F1-score, and false-positive rate) of the reduced-feature edge-ML model compared to a baseline configuration when identifying SYN-flood and SSH brute-force attacks?
- **RQ2:** How feasible is the real-time deployment of an integrated, automated detection and mitigation pipeline on resource-constrained edge hardware?
- **RQ3:** What are the latency-resource trade-offs regarding inference speed, time-to-block, and CPU/RAM utilisation when the system is subjected to simulated high-volume traffic?

1.4 Scope of Study

To ensure the project remains achievable within the given timeframe and hardware constraints, the scope is defined by the following parameters:

- **Hardware and Deployment Boundaries:** The system will be developed for and deployed on a standalone Raspberry Pi 4 Model B (4GB RAM) operating on a 64-bit Linux OS (e.g., Debian/Raspberry Pi OS). The network topology utilizes a dual-port bridging setup comprising the onboard Gigabit Ethernet port and an external USB 3.0-to-Gigabit Ethernet adapter to intercept in-line traffic. The system will be designed with a fail-open physical network bridge; if the user-space detection application fails, standard Linux kernel packet forwarding will continue to ensure downstream availability.
- **Threat Profiling and Attack Scope:** The detection parameters are strictly limited to evaluating well-defined scenarios: SYN-flood DDoS and SSH brute-force attacks. Live-fire evaluations will utilize authorised test tools (e.g., Kali Linux utilizing hping3 for high-rate SYN floods and Hydra for credential

attacks) within a strictly isolated local network testbed. The evaluation will measure specific traffic rates, session durations, and single-versus-multiple source IP scenarios.

- **Network Protocol Limits:** The packet capture, flow-aggregation, and mitigation rules (via iptables) will focus exclusively on Internet Protocol version 4 (IPv4) traffic. IPv6 traffic, encrypted application payload inspection (Deep Packet Inspection), and Layer-7 application-level attacks are explicitly outside the scope of this study.
- **Data and Machine Learning Scope:** The lightweight edge-ML model utilizes a Random Forest classifier trained on the CIC-IDS-2017 benchmark dataset. To accommodate edge processing limits, the data is reduced to eight core quantitative flow features rather than utilizing computationally heavy full-feature models.

1.5 Significance of the Project

This project provides a critical operational advantage for resource-constrained remote enterprise networks that operate on limited grid power and high-latency cellular or satellite connections. By transitioning intrusion defense to local automated detection and temporary IP-based filtering, this architecture reduces the cloud processing bottleneck.

While edge-based mitigation cannot prevent external volumetric attack traffic from consuming upstream satellite or cellular bandwidth, dropping malicious IP addresses precisely at the network demarcation point protects downstream internal corporate firewalls and field devices from rapid CPU exhaustion, as conceptualized in Figure 2. Furthermore, successfully mitigating targeted SSH brute-force attacks at the edge prevents successful credential compromise, cutting off an attacker's opportunity to execute lateral movement and further intrusion deeper into the internal network subject to internal controls. Ultimately, this empowers remote networks to proactively shield their local assets without relying on delayed remote remediation.

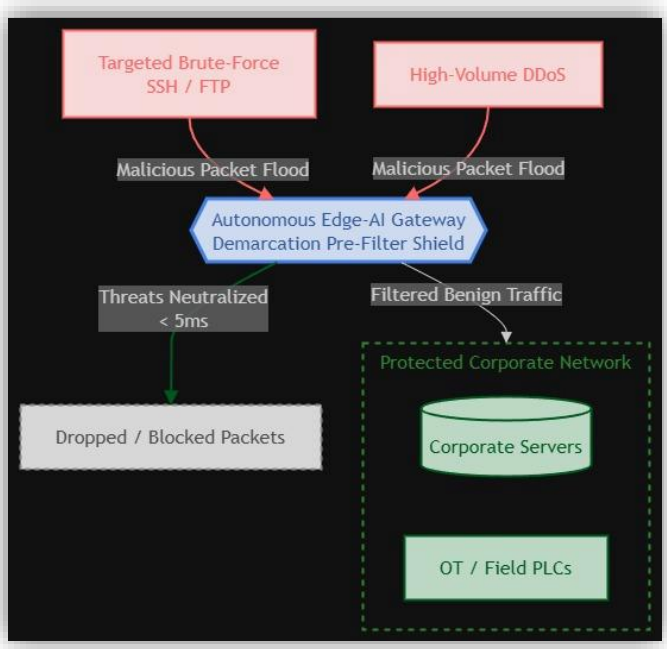


Figure 2: Conceptual Diagram of the Edge-AI Pre-Filter Shield at the Network Demarcation Point

2. LITERATURE REVIEW

2.1 The Shift from Cloud to Edge Computing in Cybersecurity

Modern Intrusion Detection System (IDS) architectures generally fall into three categories: cloud-managed systems that rely on remote servers, locally deployed systems constrained by edge hardware, and hybrid designs. Cloud-managed and hybrid setups require transmitting network data to remote infrastructure. This data can range from raw packets and flow records to extracted features or security alerts. While cloud computing offers extensive scalability, recent studies highlight its critical limitations when applied to remote enterprise environments. Rather than assuming all remote connections are inherently slow, it must be noted that these sites may depend on satellite or cellular links that can experience high latency or intermittent availability.

Cherukuri (2024) emphasizes that transmitting data to cloud servers introduces physical transmission delays, creating a vulnerability window that automated botnets can exploit. In contrast, edge computing decentralizes this process by analyzing data directly at the network perimeter. While Cherukuri notes the lower processing power of edge nodes as a limitation, the trade-off yields ultra-low latency and preserves localized data privacy. Furthermore, in high-throughput edge environments, the speed of detection is as critical as accuracy. To prevent local CPU exhaustion and network buffer overflows during volumetric attacks, an in-line mitigation framework must operate near wire-speed. This establishes an application-based rationale for targeting an inference and mitigation latency of under 5 milliseconds, ensuring computational overhead remains negligible while dropping malicious flows before upstream saturation cascades into the internal network.

2.2 Machine Learning for Traffic Analysis and Anomaly Detection

Machine Learning (ML) has become the standard for identifying complex network anomalies, but the choice of algorithm heavily dictates operational viability at the edge. Ibrahim et al. (2026) conducted a comparative evaluation of ML techniques for detecting DDoS intrusions, concluding that Random Forest (RF) significantly outperforms heavier models like Support Vector Machines (SVM) in both accuracy and execution speed. Furthermore, Manurung et al. (2026) demonstrated that

optimizing RF models with the Synthetic Minority Over-sampling Technique (SMOTE) effectively resolves class imbalances, allowing for the rapid detection of minority attack vectors like targeted brute-force attempts. However, deploying unoptimized RF models on edge devices remains challenging due to hyperparameter sensitivity and the risk of computational bloat.

Furthermore, evaluating these ML algorithms relies heavily on benchmark datasets such as CIC-IDS-2017 (Manurung et al., 2026). While comprehensive, the deployment of historical datasets in modern edge environments necessitates a critical analysis of several limiting factors. Methodologies must address the dataset's age, the potential domain shift when compared against modern heavily encrypted enterprise traffic, and the presence of duplicate flow records which can artificially inflate accuracy metrics during validation. Consequently, rigorous data cleaning and techniques such as SMOTE must be carefully applied respecting the underlying attack-session structure to handle inherent class imbalances without causing data leakage between training and testing splits.

2.2.1 Types of Machine Learning Algorithm for IDS

Machine learning models for Intrusion Detection Systems (IDS) are generally categorized into supervised, unsupervised, and hybrid approaches. Supervised algorithms, such as Support Vector Machines (SVM), Artificial Neural Networks (ANN), and Decision Trees, are commonly deployed to detect known attack signatures. However, while complex algorithms and boosting models can yield high theoretical accuracy, they frequently struggle with the strict computational resource limits and memory constraints present on edge devices (Hafid et al., 2024).

2.2.2 The Random Forest Advantages

Random Forest (RF) is an optimal choice for edge deployment because its ensemble structure, as depicted in Figure 3, balances high accuracy with computational efficiency. Ibrahim et al. (2026) showed that RF outperforms computationally heavy models like SVM and unstable architectures like ANN in both execution speed and accuracy during high-throughput DDoS attacks. Manurung et al. (2026) highlighted that RF integrates well with data-balancing techniques like SMOTE. This combination

allows for reliable detection of minority attack classes, such as targeted brute-force attempts, without the heavy processing overhead of deep learning.

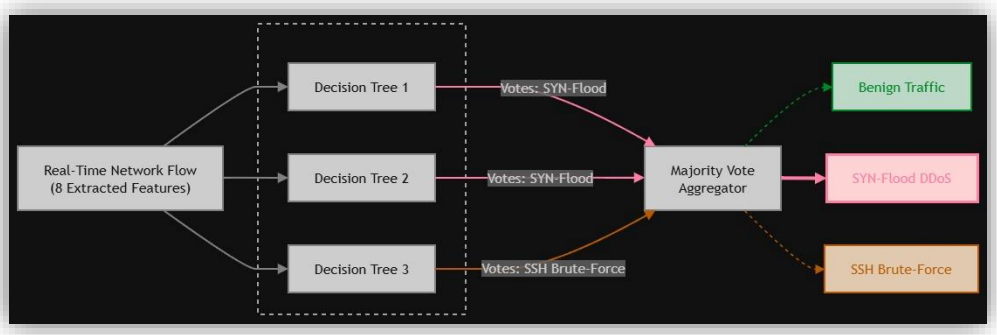


Figure 3: Random Forest Ensemble Architecture for Network Traffic Classification

2.3 Lightweight Edge Machine Learning in Edge Security

2.3.1 Definition and Industry Applications of Lightweight Edge-ML

Historically, "TinyML" has been strictly defined as the deployment of machine learning models on extreme ultra-low-power microcontrollers using recognized model conversion frameworks. However, modern remote security requirements necessitate a shift towards "lightweight edge machine learning" (edge-ML). This paradigm adapts model compression, feature reduction, and algorithmic optimization techniques and scales them for higher-tier edge devices, such as the Raspberry Pi 4. By utilizing lightweight edge-ML rather than conventional server-grade ML, organizations can execute rapid inference directly at the network demarcation point without the power consumption or computational bloat associated with traditional systems.

2.3.2 Comparative Analysis of Lightweight Edge-ML with other Traditional Edge ML

Traditional edge ML often attempts to deploy full-scale deep learning models at the network perimeter. As demonstrated by Ali et al. (2025), deep transfer learning can be applied to an IoT edge IDS. However, this approach demands heavy initial GPU compute resources, and model synchronization consumes valuable network bandwidth. Similarly, integrating Explainable AI (XAI) adds significant computational complexity that limits real-time scalability (Mohale & Obagbuwa, 2025). In contrast,

as compared in Figure 4, lightweight edge-ML relies on rigorous feature selection and algorithmic compression to execute rapid local inference. By reducing the feature set to only the most critical flow parameters, lightweight edge-ML avoids the energy bottlenecks and CPU exhaustion inherent in traditional edge deployments (Amuthadevi et al., 2025).

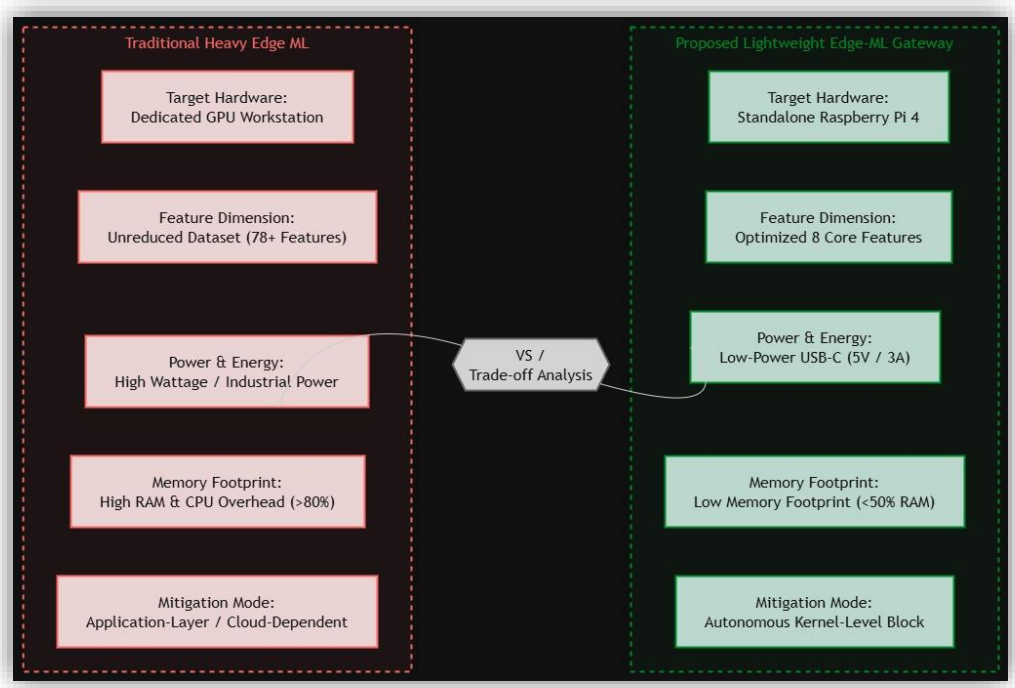


Figure 4: Traditional Deep Learning Edge Deployments vs. Lightweight Edge-ML

2.4 Resource-Constrained Remote Enterprise Networks

Remote enterprise environments present unique physical and digital operational challenges that separate them from standard corporate networks. Security frameworks deployed in these zones must be engineered to respect strict limitations regarding power consumption, hardware footprint, and external connectivity (Zhu & Jiang, 2022).

2.4.1 Characteristics and Examples

Prime examples of these environments include offshore platforms and isolated agricultural facilities. These sites frequently rely on satellite or cellular links for external communication. While often assumed to be universally slow, it is more

accurate to state that these links may experience high latency, limited bandwidth, or intermittent availability depending on environmental and operational factors (Zhu & Jiang, 2022). Consequently, cybersecurity architectures in these environments cannot assume continuous, high-speed connectivity to cloud-based management servers, reinforcing the necessity for localized, autonomous threat mitigation.

2.5 Comparative Summary of Reviewed Studies

Table 1 summarizes the technology focus, strengths, weaknesses, and limitations of the seven studies reviewed in this chapter, grouped respectively by their emphasis on edge/cloud architecture and ML algorithm choice, and on lightweight edge deployment.

Table 1: Comparative Summary of Reviewed Studies

Study	Technology/Focus	Strengths	Weaknesses	Limitations
Cherukuri (2024)	Edge vs. cloud computing for real-time AI	Edge: ultra-low latency and local data privacy. Cloud: massive scalability and high processing speed.	Edge: limited scalability and lower processing power. Cloud: high latency and data-transit security risks.	Edge: risk of physical tampering and difficulty managing distributed nodes. Cloud: high recurring costs based on data volume.
Manurung et al. (2026)	Optimised Random Forest with SMOTE and feature selection for IDS	High accuracy and strong minority-attack detection; fast training.	Highly sensitive to hyperparameter tuning; unoptimised baseline model fails on minority classes.	Evaluated on a single dataset (CIC-IDS-2017); requires future testing in real-time operational environments.
Ibrahim et al. (2026)	Comparative evaluation of ML techniques for detecting DDoS intrusions in IoT	Random Forest excels in both accuracy and speed.	SVM is computationally heavy; ANN lacks stability.	Evaluated only four attack types; no physical IoT deployment.

Amuthadevi et al. (2025)	Lightweight Edge-ML for ultra-low-power IoT IDS	High accuracy and fast processing; minimal energy and memory usage.	High initial deployment costs; energy bottlenecks during high-frequency attacks.	Restricted data transfer in ultra-low-power setups; requires future model-compression enhancements
Ali et al. (2025)	Deep transfer learning for IoT edge IDS	High accuracy; ultra-lightweight "supermodel" ideal for IoT.	Initial training demands heavy GPU compute resources; model synchronisation consumes network bandwidth.	Tested on a generic dataset; vulnerable to adversarial and poisoning attacks.
Hafid et al. (2024)	Boosting ML models for IoT edge IDS	High accuracy and F1-score; excellent iterative error correction for complex threats.	MLP and Decision Trees perform poorly on complex data; LightGBM shows lower accuracy.	Edge device resource limits complicate deployment.
Mohale & Obagbuwa (2025)	ML models with Explainable AI (XAI) for IDS	High accuracy balanced with transparency; actionable insights via feature importance.	XAI integration adds high computational complexity; risk of false negatives on advanced threats.	Evaluated solely on the UNSW-NB15 dataset; heavy compute needs limit real-time scalability.

2.6 Critical Analysis and Identified Research Gaps

A synthesis of current literature reveals a persistent disconnect between theoretical ML accuracy and practical edge deployment. Studies by Hafid et al. (2024) and Mohale & Obagbuwa (2025) focused on boosting model accuracy and integrating Explainable AI (XAI) into IDS frameworks. While these approaches yield excellent theoretical

insights and high F1-scores, the added computational complexity restricts real-time scalability on devices like the Raspberry Pi.

Four research gaps emerge consistently across the reviewed literature:

1. **Heavy Computational Demands:** Current high-accuracy models require massive GPU resources and memory, making them largely unviable for deployment on resource-constrained edge devices such as the Raspberry Pi.
2. **Cloud-Dependent Latency:** Existing architectures still rely on off-site cloud servers for heavy packet analysis, introducing network latency and a dangerous window where a successful credential compromise may create an opportunity for further intrusion, subject to network controls, before a response is triggered.
3. **Lack of Autonomous Mitigation:** Most edge-based IDS research focuses almost exclusively on detection and alerting. There is a distinct lack of frameworks that seamlessly bridge edge-native threat detection with instantaneous, offline, and autonomous mitigation, such as the execution of kernel-level iptables commands.
4. **Real-Time Feature Extraction Bottleneck:** While existing literature frequently boasts high accuracy using datasets like CIC-IDS-2017, a critical research gap remains regarding real-time feature extraction. In a live edge deployment, a standalone gateway must actively maintain bidirectional flow states, timeouts, packet counts, packet-length statistics, and TCP flags on the fly to accurately reproduce the dataset's feature definitions locally. Existing studies frequently omit the severe processing overhead required for this real-time flow aggregation, highlighting the need for architectures that bridge the gap between static CSV dataset training and live packet interception.

This project directly addresses these gaps by **developing** a closed-loop Edge-AI gateway that not only detects threats but also autonomously executes local firewall mitigation without human intervention or cloud dependency.

2.7 Chapter Summary

This chapter reviewed literature across three intersecting areas: edge versus cloud computing architectures, machine learning approaches to traffic anomaly detection,

and lightweight edge machine learning for resource-constrained devices. The review established that Random Forest is a strong candidate algorithm for edge deployment, that SMOTE and feature reduction can address class imbalance and computational load respectively, and that no reviewed study fully closes the loop between detection and autonomous, offline mitigation. These findings directly motivate the proposed methodology described in Chapter 3.

3. METHODOLOGY

This chapter presents the proposed methodology for the Autonomous Edge-AI gateway, utilizing iterative prototyping for system development. This planned research design will be implemented and validated during FYP2, structured around the project phases identified in the Gantt charts in Appendix A and Appendix B.

3.1 Software Development Life Cycle (SDLC)

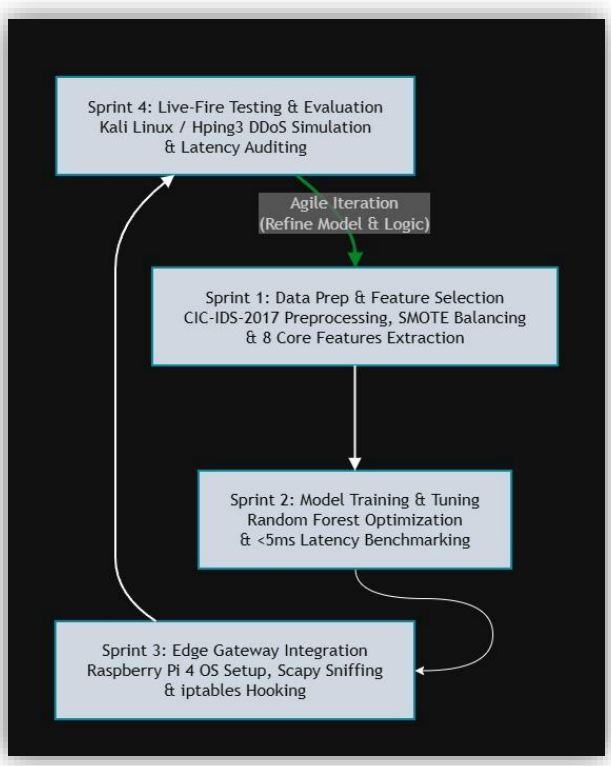


Figure 5: Adapted Agile Software Development Life Cycle for Edge-ML Integration

3.1.1 Overview of the Agile Methodology

This project utilizes an Agile Software Development Life Cycle (SDLC) centered on iterative prototyping, as illustrated in Figure 5. This approach allows for the continuous deployment, testing, and refinement of the machine learning model and gateway networking components through successive developmental sprints, ensuring that hardware constraints are identified and mitigated early.

3.1.2 Application of SDLC Phases to the Project

Embedded within the Agile SDLC framework is a separate, rigorous experimental methodology tailored specifically for edge-machine learning. This covers strict protocols for data preparation, model development, system integration, controlled testing, and statistical analysis, ensuring that the engineering tasks mapped in the project phases translate into scientifically valid evaluations.

3.2 Phase 1: Planning and System Architecture

3.2.1 Overall System Architecture and Network Demarcation

The gateway will be deployed in line at the network demarcation point. The system architecture diagram (Figure 6) details the complete traffic path as it passes through the bridge interfaces. Internally, the software architecture is designed to incorporate a flow table for network state management, a real-time feature extractor, the lightweight machine learning decision engine, and a firewall-rule manager, supported by event logging and an optional administrative dashboard.

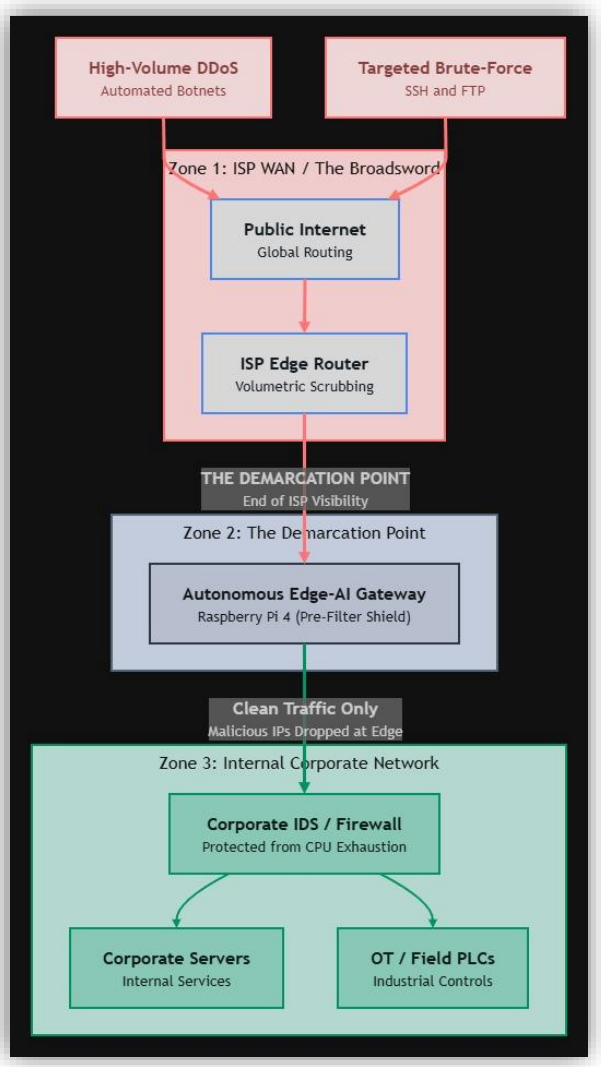


Figure 6: Multi-Zone Network Demarcation and Edge Gateway System Architecture

3.2.2 Hardware and Software

The system will be exclusively deployed on a Raspberry Pi 4 Model B equipped with 4GB of RAM and running a 64-bit Linux operating system. To intercept in-line traffic, the setup will utilize the onboard Gigabit Ethernet port paired with an external USB 3.0-to-Gigabit Ethernet adapter, ensuring maximum theoretical interface speeds. Crucially, the physical topology is engineered to fail-open; the architecture ensures that standard Linux kernel traffic forwarding continues unabated if the user-space detection application fails, preserving downstream network availability.

3.3 Phase 2: Design and Implementation

3.3.1 Dataset Acquisition and Feature Engineering

A formal feature-selection procedure will be conducted to identify a heavily reduced set of eight core features, providing evidence that these specific features are consistently available for benign, SYN-flood DDoS, and SSH brute-force flows. To transition from dataset training to live edge inference, a real-time flow-aggregation component will be designed, as detailed in Figure 7. This component will actively maintain bidirectional flow states, timeouts, packet counts, packet-length statistics, inter-arrival times, and TCP flags. This ensures the live-extracted definitions strictly match the CIC-IDS-2017 feature extractor.

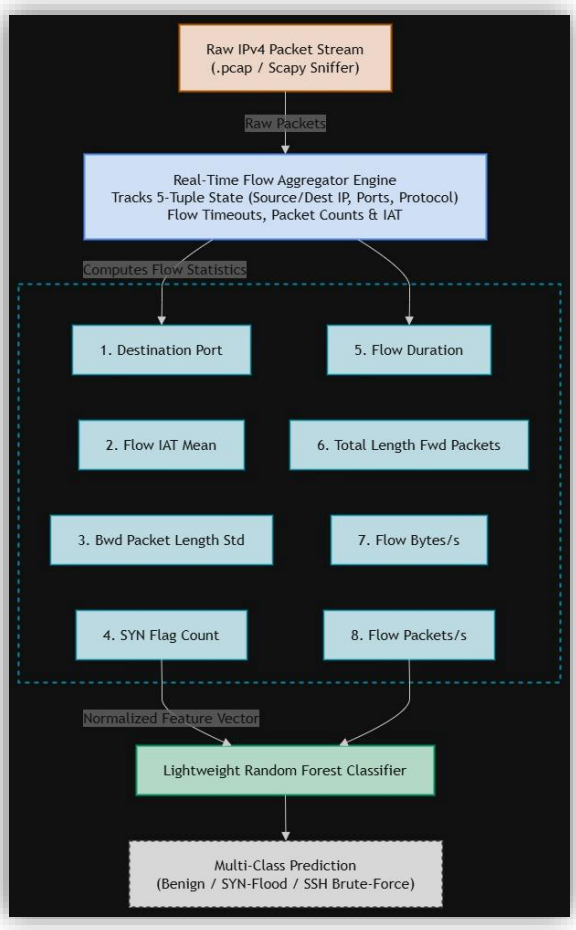


Figure 7: Real-Time Flow Aggregation and Feature Extraction Pipeline

3.3.2 Model Development and Baseline Benchmarking

To prevent data leakage, the dataset will be split into training and validation sets strictly by attack session, capture period, or source IP. Random row splitting will be avoided entirely so that records from the same session do not appear in both sets. The Synthetic Minority Over-sampling Technique (SMOTE) will be applied exclusively to the training partition to resolve class imbalances. Hyperparameter optimization will rely on a defined search method and validation procedure to tune key parameters, such as the number of trees, maximum depth, minimum samples, and class weighting, while reporting the final model size. The model output will clearly define the attack classes and the established decision threshold. Finally, the reduced-feature Random Forest model will be evaluated against a full-feature baseline and at least one simpler model, such as a Decision Tree or Logistic Regression.

3.3.3 Edge Integration and Automated Firewall Rules

The autonomous IP mitigation module will implement temporary iptables rules configured with automated expiry, rollback mechanisms, a strict maximum rule count, comprehensive logging, and manual administrator overrides, as illustrated in the mitigation framework (Figure 8). To prevent self-inflicted network isolation, a hardcoded allowlist will be enforced, ensuring that trusted gateways, DNS servers, management hosts, and the Raspberry Pi itself cannot be accidentally blocked.

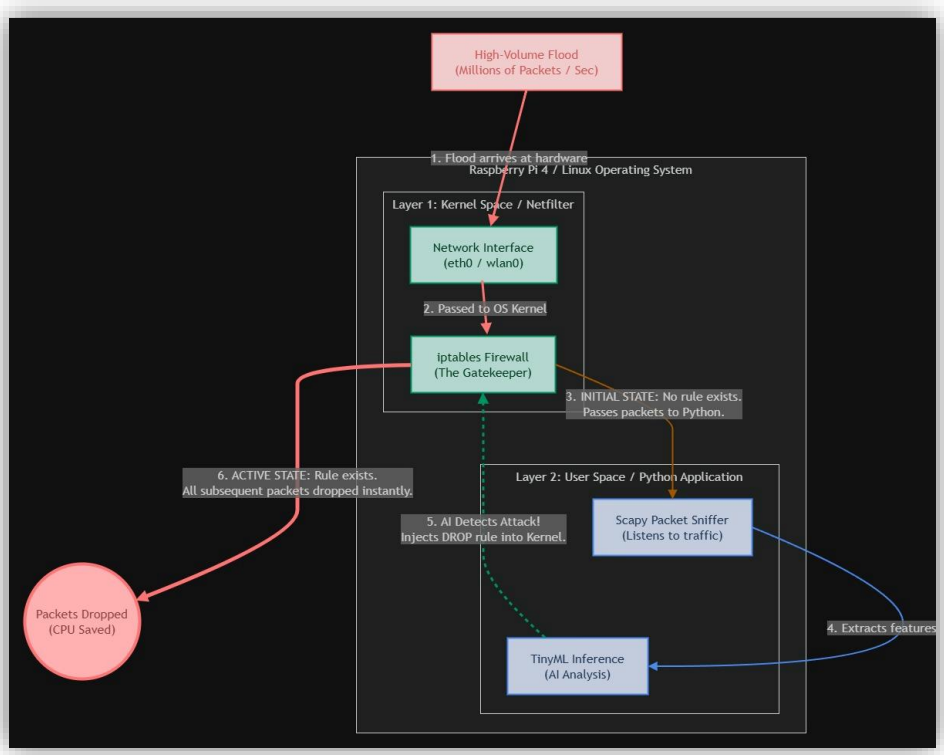


Figure 8: Closed-Loop Linux Kernel Mitigation and Packet Inspection Execution Flow

3.4 Phase 3: Testing and Evaluation Plan

3.4.1 Threat Simulation Scenarios

All live-fire tests will take place in a strictly isolated, authorized network testbed. The simulation setup will specify the traffic generation tools (such as hping3 and Hydra), rate limits, network isolation rules, and logging protocols, including safeguard procedures to ensure attack traffic never reaches production or public networks. Additionally, standard background traffic, including web browsing, SSH management, file transfers, and DNS requests, will run alongside the attack simulations to evaluate the model's false-positive rate under realistic conditions.

3.4.2 Performance Metrics

The evaluation will compare local edge-native mitigation latency against remote, cloud-based end-to-end mitigation latency under identical traffic conditions. To guarantee statistical validity, each simulated traffic condition will be run multiple

times. The final evaluation will rigorously report the mean, median, standard deviation, and confidence intervals for all hardware resource consumption and latency metrics.

3.5 Project Timeline

The detailed week-by-week schedule for the work completed in FYP1 and the work planned for FYP2 is presented in Appendix A and Appendix B respectively.

4. EXPECTED OUTCOMES

4.1 Work Completed in FYP1

4.1.1 Dataset Preparation and Feature Reduction

During FYP1, the CIC-IDS-2017 dataset was acquired, and a preliminary analysis of its attack-session structure was conducted. The theoretical foundation for a formal feature-selection procedure was established to identify the eight core quantitative network flow parameters required to accurately differentiate benign traffic from SYN-flood DDoS and SSH brute-force attacks without relying on full-feature extraction.

4.1.2 Architecture Design and SDLC Planning

The overall edge gateway architecture was successfully designed, detailing both the fail-open physical network topology for the Raspberry Pi 4 and the internal software traffic path. Additionally, an experimental methodology was integrated into the Agile SDLC to guide the upcoming FYP2 implementation phases. This methodology defines strict dataset splitting rules, hardware constraints, and ethical testbed isolation protocols.

5. CONCLUSION AND RECOMMENDATIONS

5.1 Summary

This report outlined the proposed development of an autonomous edge machine learning gateway designed to resolve key issues with cloud-dependent latency, high computational demands, and limited ISP visibility in remote enterprise networks. By using lightweight edge ML and automated local IP mitigation, this project aims to provide an active, resource-efficient pre-filter shield positioned directly at the network demarcation point.

5.2 Conclusion of FYP1

During FYP1, the theoretical foundation, physical network topology, and overall system architecture for the project were successfully established. A comprehensive literature review identified key research gaps, particularly the lack of real-time feature extraction and closed-loop automated mitigation frameworks designed for resource-constrained hardware. To address these gaps, a rigorous experimental methodology was created, incorporating strict dataset splitting rules, hardware fail-open safety measures, and authorized threat simulation protocols.

5.3 Project Continuation in FYP2

FYP2 will focus entirely on executing the established experimental methodology to deliver three core outcomes. First, a reduced-feature Random Forest model will be trained and optimized on SMOTE-balanced data, ensuring high classification performance on resource-constrained hardware while keeping the final model size minimal. Second, the autonomous in-line gateway will be deployed on the Raspberry Pi by building a real-time flow-aggregation engine and connecting it to an automated netfilter/iptables mitigation loop with temporary rules and hardcoded allowlists. Finally, the completed prototype will undergo rigorous evaluation in an isolated testbed. This testing will statistically quantify detection accuracy, CPU and RAM utilization, and execution latency, as conceptualized in the proposed monitoring dashboard (Figure 9), ultimately determining if the gateway can meet its sub-5-millisecond target under active attack conditions.

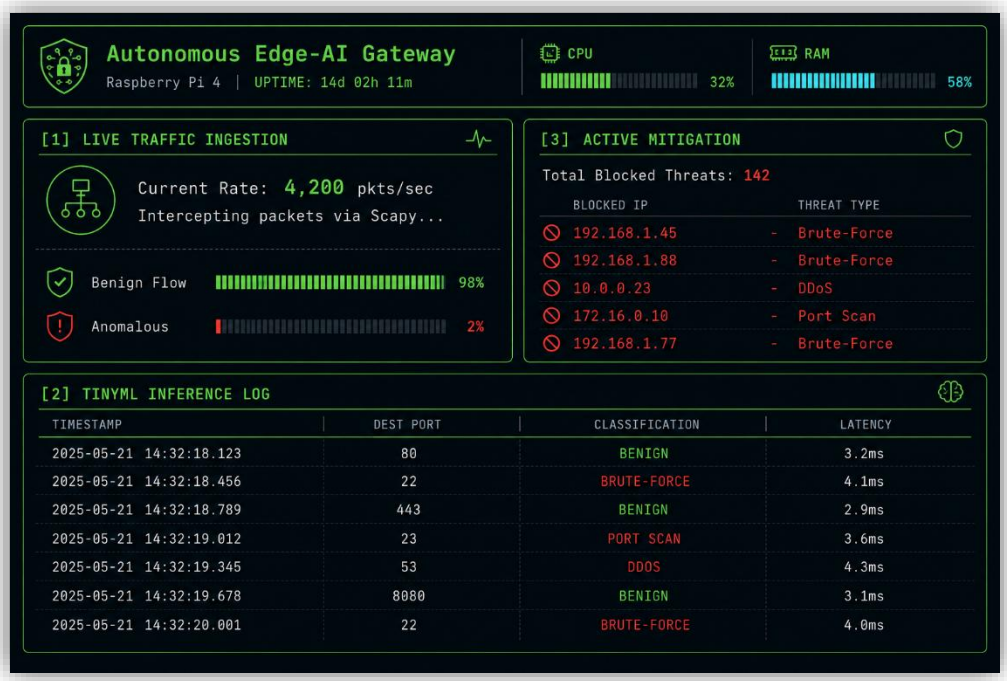


Figure 9: Conceptual Edge-ML Gateway Dashboard Mockup for Real-Time Telemetry and Mitigation

5.4 Recommendation for Future Work

The planned evaluation aims to establish whether edge-native automated filtering is a viable, high-speed replacement for delayed remote remediation. However, while the proposed framework is anticipated to protect downstream services and reduce local processing loads, upstream network saturation remains an acknowledged limitation. Future work beyond the scope of this project should explore integrating the edge gateway with automated ISP-level APIs to trigger upstream volumetric scrubbing once a local attack is detected. Additionally, future iterations could expand the detection capabilities to encompass IPv6 traffic and deep packet inspection for application-layer attacks.

REFERENCES

Ali, Z., Tiberti, W., Marotta, A., & Cassioli, D. (2025). Deep transfer learning for intrusion detection in edge computing scenarios. *IEEE Internet of Things Journal*, *12*(21), 44882 44896. https://doi.org/10.1109/JIOT.2025.3597892 [cite: 7]

Amuthadevi, C., Venkatesan, R., Mythily, M., & Aroul Canessane, R. O. (2025). TinyML-based intrusion detection systems for sustainable and energy-constrained IoT devices. *Results in Engineering*, *28*, Article 108013. https://doi.org/10.1016/j.rineng.2025.108013 [cite: 8]

Aryee, E. N. A. (2026). *From packets to pixels: A comparative analysis of machine learning and image-based deep learning for encrypted traffic classification* (Publication No. 32676860) [Master's thesis, University of Michigan-Flint]. ProQuest Dissertations and Theses Global. [cite: 7]

Cherukuri, B. R. (2024). Edge computing vs. cloud computing: A comparative analysis for real time AI applications. *International Journal for Multidisciplinary Research (IJFMR)*, 6(5), 1–17. https://doi.org/10.58212/ijfmr.2024.v06i05.29316

Hafid, B., Ezzouhairi, A., & Haddouch, K. (2024). Strengthening security in the Internet of Things (IoT): Integrated approach of intrusion detection systems (IDS) and edge computing. In *2024 3rd International Conference on Embedded Systems and Artificial Intelligence (ESAI)* (pp. 1–9). IEEE. https://doi.org/10.1109/ESAI62891.2024.10913549

Ibrahim, A., Imam, N. M., Wakili, A. A., & Emmanuel, M. (2026). Performance evaluation of machine learning techniques for network intrusion detection in IoT systems. *IOSR Journal of Computer Engineering*, *28*(2), 81–89. https://doi.org/10.9790/06612802018189 [cite: 10]

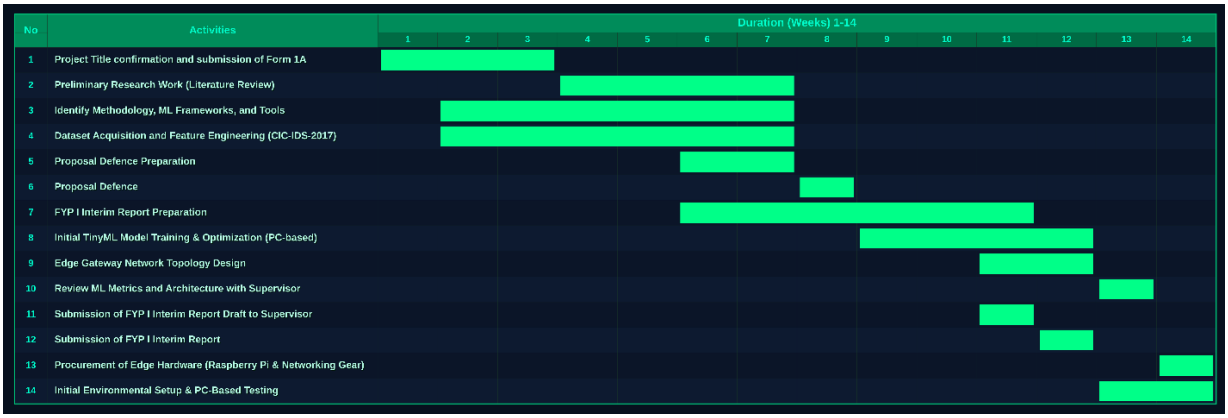
Manurung, J., Mardamsyah, A., & Sianipar, B. (2026). Enhanced cyber attack detection using optimized random forest with SMOTE-based class balancing and feature selection. *Journal of Defense Technology and Engineering*, *1*(2), 65–80.

Mohale, V. Z., & Obagbuwa, I. C. (2025). Evaluating machine learning-based intrusion detection systems with explainable AI: Enhancing transparency and interpretability. *Frontiers in Computer Science*, *7*, Article 1520741. https://doi.org/10.3389/fcomp.2025.1520741 [cite: 12]

Zhu, X., & Jiang, C. (2022). Integrated satellite-terrestrial networks toward 6G: Architectures, applications, and challenges. *IEEE Internet of Things Journal*, *9*(1), 437–461. https://doi.org/10.1109/JIOT.2021.3126825 [cite: 13]

APPENDICES

APPENDIX A: FYP1 Project Gantt Chart



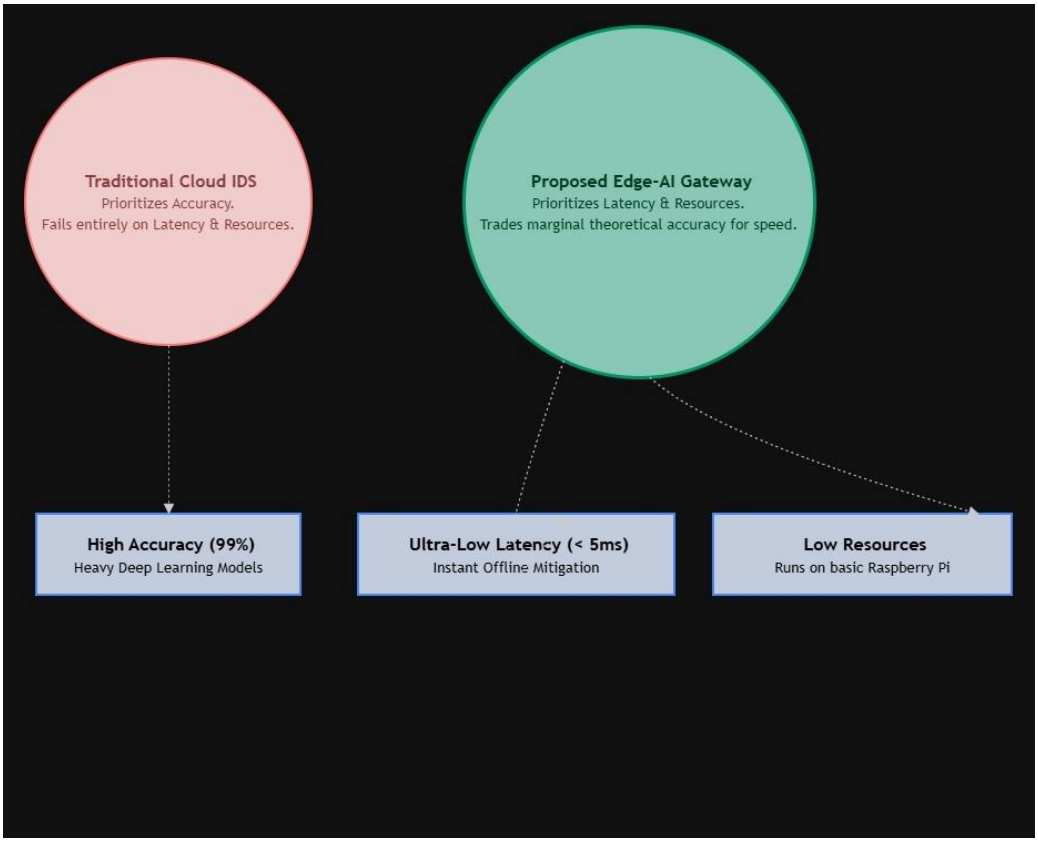
Appendix A illustrates the week-by-week timeline and task completion schedule for Final Year Project 1 (FYP1) spanning Weeks 1 through 14. It details foundational activities including title selection, literature synthesis, model selection, dataset acquisition, proposal defense, and initial PC-based machine learning training.

APPENDIX B: FYP2 Project Gantt Chart



Appendix B outlines the operational milestone schedule planned for Final Year Project 2 (FYP2) across 14 developmental weeks. It maps technical milestones including Raspberry Pi edge setup, real-time feature extraction implementation, iptables mitigation pipeline integration, live-fire attack simulations in an isolated testbed, latency benchmarking, and final dissertation preparation.

APPENDIX C: Trade-Off Conceptual Model: Edge-AI vs. Cloud IDS



Appendix C illustrates the architectural trade-off analysis comparing traditional heavy cloud-managed IDSs with the proposed autonomous Edge-AI gateway. While traditional cloud IDSs prioritize theoretical accuracy (~99%) using heavy deep learning models at the cost of high transmission latency and resource reliance, the proposed Edge-AI gateway trades a marginal fraction of theoretical accuracy for sub-5-millisecond local mitigation speed, minimal memory overhead, and offline independence on low-cost hardware.